

Project Report

Automata & Formal Languages Engine

Submitted by: Dhamat Yaksh M.

Enrollment Number: 230210107012

Technologies Used:

Java 24, Spring Boot 3.3.0, Hibernate ORM, MySQL, JavaScript

1. Introduction

The Automata & Formal Languages Engine is a full-stack web application designed to visually create, mathematically model, and computationally simulate Deterministic Finite Automata (DFA). This project bridges the gap between theoretical computer science concepts and practical software engineering, providing an interactive platform to test strings against custom-built state machines.

The primary objective of this project was to implement a robust $O(n)$ simulation algorithm backed by a strict relational database schema that enforces mathematical determinism at the data layer, ensuring that invalid or non-deterministic structures cannot exist within the system.

2. Mathematical Foundation

The core of the simulation engine is based on the formal mathematical definition of a Deterministic Finite Automaton. A DFA is defined as a 5-tuple, $\mathbf{M} = (\mathbf{Q}, \Sigma, \delta, \mathbf{q_0}, \mathbf{F})$, where:

- $\mathbf{Q}$: A finite set of states (represented by the `State` entity).
- Σ : A finite set of input symbols called the alphabet (stored within the `Machine` entity).
- δ : The transition function $\delta: \mathbf{Q} \times \Sigma \rightarrow \mathbf{Q}$ (represented by the `Transition` entity).
- $\mathbf{q_0}$: The initial or start state, where $\mathbf{q_0} \in \mathbf{Q}$.
- $\mathbf{F}$: A set of accept or final states, where $\mathbf{F} \subseteq \mathbf{Q}$.

3. System Architecture

The application follows a modern, three-tier architectural pattern, isolating data, business logic, and presentation layers.

Layer	Technology Stack	Responsibility
Frontend (Presentation)	Vanilla HTML5, CSS3, JavaScript (Fetch API)	Provides a minimalist UI for drawing the DFA structure and displaying the trace path of the string evaluation.
Backend (Application)	Java 24, Spring Boot 3.3.0, Spring Web	Hosts the REST API endpoints and executes the $O(n)$ <code>MachineService</code> simulation algorithm.
Database (Persistence)	MySQL, Spring Data JPA (Hibernate)	Persists machines, states, and transitions while enforcing mathematical integrity constraints.

4. Database Schema & Integrity

To ensure the system strictly simulates Deterministic Finite Automata (and prevents accidental Non-Deterministic Finite Automata - NFA), a critical unique constraint was engineered into the database schema using Hibernate annotations.

```
ALTER TABLE transition ADD CONSTRAINT UK_transition_rule UNIQUE
(machine_id, from_state_id, input_symbol);
```

This guarantees the definition of determinism: for any given state and any given input symbol, there can be *at most one* valid transition path. If a user attempts to create two divergent paths for the same input symbol leaving the same state, the database physically rejects the transaction.

5. Core Simulation Algorithm

The simulation logic is encapsulated within the `MachineService.java` component. The algorithm achieves an optimal time complexity of $O(n)$, where n is the length of the input string.

Algorithmic Steps:

1. **Initialization:** Locate the defined start state (q_0). If none exists, abort.

2. **Traversal Loop:** For each character in the input string sequence:

- Verify the character exists in the machine's defined alphabet (Σ). If not, halt and return **REJECTED** (Invalid Symbol).
- Query the transition set for an edge matching the current state and the current input symbol.
- If a transition is found, update the current state pointer.
- If no transition is found, halt and return **REJECTED** (Dead End).

3. **Resolution:** Upon consuming the entire string, evaluate the final state pointer. If the final state is a member of the accepting states set (F), return **ACCEPTED**; otherwise, return **REJECTED**.

6. Edge Cases & Testing Scenarios

The engine was thoroughly tested against several standard computational edge cases to verify mathematical accuracy:

1. The Epsilon (ϵ) Case:

When evaluated with an empty string, the $O(n)$ traversal loop is entirely bypassed. The engine immediately evaluates the initial state q_0 . If q_0 is an accepting state, the empty string is accepted; otherwise, it is rejected.

2. The Trap / Absorbing State:

Machines were constructed containing states from which no transition leads out, except self-loops. Once the traversal enters this state, it successfully remains locked until the string is consumed.

3. Missing Transitions (Implicit Dead States):

Instead of requiring the user to manually draw paths to a "trash state" for every invalid input, the engine intelligently handles `null` next-state resolutions by instantly halting the simulation and rejecting the string, saving computational cycles.

7. Conclusion

Developing the Automata & Formal Languages Engine provided profound insights into both theoretical computer science and modern enterprise software engineering. By mapping the

abstract mathematical concepts of DFA tuples directly into Object-Relational Mapping (ORM) entities, the project demonstrated how mathematical constraints can be elegantly enforced at the database level.

Furthermore, overcoming the integration challenges between the JSON frontend and the bi-directional JPA backend solidified a strong understanding of Spring Boot's data binding and RESTful API design. The resulting application is a highly scalable, mathematically sound educational tool.